# ClangFormat

配置文档

*The Clang Team*

*Copyright © The Clang Team.*

# 目录


1. 使用 6
1.1 介绍 6
1.2 配置样式 8
1.3 禁用格式 9
2. 配置 10
2.1 BasedOnStyle 10
2.2 AccessModifierOffset 11
2.3 AlignAfterOpenBracket 12
2.4 AlignArrayOfStructures 13
2.5 AlignConsecutiveAssignments 14
2.6 AlignConsecutiveBitFields 15
2.7 AlignConsecutiveDeclarations 16
2.8 AlignConsecutiveMacros 17
2.9 AlignEscapedNewlines 18
2.10 AlignOperands 19
2.11 AlignTrailingComments 20
2.12 AllowAllArgumentsOnNextLine 21
2.13 AllowAllConstructorInitializersOnNextLine 22
2.14 AllowAllParametersOfDeclarationOnNextLine 23
2.15 AllowShortBlocksOnASingleLine 24
2.16 AllowShortCaseLabelsOnASingleLine 25
2.17 AllowShortEnumsOnASingleLine 26
2.18 AllowShortFunctionsOnASingleLine 27
2.19 AllowShortIfStatementsOnASingleLine 28
2.20 AllowShortLambdasOnASingleLine 30
2.21 AllowShortLoopsOnASingleLine 31
2.22 AlwaysBreakAfterDefinitionReturnType 32
2.23 AlwaysBreakAfterReturnType 33
2.24 AlwaysBreakBeforeMultilineStrings 35
2.25 AlwaysBreakTemplateDeclarations 36
2.26 AttributeMacros 37
2.27 BinPackArguments 38
2.28 BinPackParameters 39
2.29 BitFieldColonSpacing 40
2.30 BraceWrapping 41



Copyright © The Clang Team.

2.31 BreakAfterJavaFieldAnnotations 46
2.32 BreakBeforeBinaryOperators 47
2.33 BreakBeforeBraces 48
2.34 BreakBeforeConceptDeclarations 52
2.35 BreakBeforeTernaryOperators 53
2.36 BreakConstructorInitializers 54
2.37 BreakInheritanceList 55
2.38 BreakStringLiterals 56
2.39 ColumnLimit 57
2.40 CommentPragmas 58
2.41 CompactNamespaces 59
2.42 ConstructorInitializerAllOnOneLineOrOnePerLine 60
2.43 ConstructorInitializerIndentWidth 61
2.44 ContinuationIndentWidth 62
2.45 Cpp11BracedListStyle 63
2.46 DeriveLineEnding 64
2.47 DerivePointerAlignment 65
2.48 DisableFormat 66
2.49 EmptyLineAfterAccessModifier 67
2.50 EmptyLineBeforeAccessModifier 68
2.51 ExperimentalAutoDetectBinPacking 69
2.52 FixNamespaceComments 70
2.53 ForEachMacros 71
2.54 IfMacros 72
2.55 IncludeBlocks 73
2.56 IncludeCategories 74
2.57 IncludeIsMainRegex 75
2.58 IncludeIsMainSourceRegex 76
2.59 IndentAccessModifiers 77
2.60 IndentCaseBlocks 78
2.61 IndentCaseLabels 79
2.62 IndentExternBlock 80
2.63 IndentGotoLabels 81
2.64 IndentPPDirectives 82
2.65 IndentRequires 83
2.66 IndentWidth 84
2.67 IndentWrappedFunctionNames 85
2.68 InsertTrailingCommas 86


Copyright © The Clang Team.

2.69 JavaImportGroups 87
2.70 JavaScriptQuotes 88
2.71 JavaScriptWrapImports 89
2.72 KeepEmptyLinesAtTheStartOfBlocks 90
2.73 LambdaBodyIndentation 91
2.74 Language 92
2.75 MacroBlockBegin 93
2.76 MacroBlockEnd 94
2.77 MaxEmptyLinesToKeep 95
2.78 NamespaceIndentation 96
2.79 NamespaceMacros 97
2.80 ObjCBinPackProtocolList 98
2.81 ObjCBlockIndentWidth 99
2.82 ObjCBreakBeforeNestedBlockParam 100
2.83 ObjCSpaceAfterProperty 101
2.84 ObjCSpaceBeforeProtocolList 102
2.85 PPIndentWidth 103
2.86 PackConstructorInitializers 104
2.87 PenaltyBreakAssignment 105
2.88 PenaltyBreakBeforeFirstCallParameter 106
2.89 PenaltyBreakComment 107
2.90 PenaltyBreakFirstLessLess 108
2.91 PenaltyBreakString 109
2.92 PenaltyBreakTemplateDeclaration 110
2.93 PenaltyExcessCharacter 111
2.94 PenaltyIndentedWhitespace 112
2.95 PenaltyReturnTypeOnItsOwnLine 113
2.96 PointerAlignment 114
2.97 QualifierAlignment 115
2.98 QualifierOrder 116
2.99 RawStringFormats 117
2.100 ReferenceAlignment 118
2.101 ReflowComments 119
2.102 ShortNamespaceLines 120
2.103 SortIncludes 121
2.104 SortJavaStaticImport 122
2.105 SortUsingDeclarations 123
2.106 SpaceAfterCStyleCast 124


Copyright © The Clang Team.

2.107 SpaceAfterLogicalNot 125
2.108 SpaceAfterTemplateKeyword 126
2.109 SpaceAroundPointerQualifiers 127
2.110 SpaceBeforeAssignmentOperators 128
2.111 SpaceBeforeCaseColon 129
2.112 SpaceBeforeCpp11BracedList 130
2.113 SpaceBeforeCtorInitializerColon 131
2.114 SpaceBeforeInheritanceColon 132
2.115 SpaceBeforeParens 133
2.116 SpaceBeforeParensOptions 134
2.117 SpaceBeforeRangeBasedForLoopColon 135
2.118 SpaceBeforeSquareBrackets 136
2.119 SpaceInEmptyBlock 137
2.120 SpaceInEmptyParentheses 138
2.121 SpacesBeforeTrailingComments 139
2.122 SpacesInAngles 140
2.123 SpacesInCStyleCastParentheses 141
2.124 SpacesInConditionalStatement 142
2.125 SpacesInContainerLiterals 143
2.126 SpacesInLineCommentPrefix 144
2.127 SpacesInParentheses 145
2.128 SpacesInSquareBrackets 146
2.129 Standard 147
2.130 StatementAttributeLikeMacros 148
2.131 StatementMacros 149
2.132 TabWidth 150
2.133 TypenameMacros 151
2.134 UseCRLF 152
2.135 UseTab 153
2.136 WhitespaceSensitiveMacros 154


Copyright © The Clang Team.

# 1. 使用

## 1.1 介绍

clang-format 位于 `clang/tools/clang-format`，可用于格式化 `C` / `C++` / `Java` / `JavaScript` / `Objective-C` / `Protobuf` / `C#` 代码。

```
$ clang-format -help
OVERVIEW: A tool to format C/C++/Java/JavaScript/JSON/Objective-C/Protobuf/C# code.

If no arguments are specified, it formats the code from standard input
and writes the result to the standard output.
If <file>s are given, it reformats the files. If -i is specified
together with <file>s, the files are edited in-place. Otherwise, the
result is written to the standard output.

USAGE: clang-format [options] [<file> ...]

OPTIONS:

Clang-format options:

  --Werror                    - If set, changes formatting warnings to errors
  --Wno-error=<value>         - If set don't error out on the specified warning type.
    =unknown                  -   If set, unknown format options are only warned about.
                                  This can be used to enable formatting, even if the
                                  configuration contains unknown (newer) options.
                                  Use with caution, as this might lead to dramatically
                                  differing format depending on an option being
                                  supported or not.
  --assume-filename=<string> - Override filename used to determine the language.
                               When reading from stdin, clang-format assumes this
                               filename to determine the language.
  --cursor=<uint>             - The position of the cursor when invoking
                               clang-format from an editor integration
  --dry-run                   - If set, do not actually make the formatting changes
  --dump-config               - Dump configuration options to stdout and exit.
                               Can be used with -style option.
  --fallback-style=<string>  - The name of the predefined style used as a
                               fallback in case clang-format is invoked with
                               -style=file, but can not find the .clang-format
                               file to use.
                               Use -fallback-style=none to skip formatting.
  --ferror-limit=<uint>      - Set the maximum number of clang-format errors to
                               emit before stopping (0 = no limit). Used only
                               with --dry-run or -n
  -i                          - Inplace edit <file>s, if specified.
  --length=<uint>             - Format a range of this length (in bytes).
                               Multiple ranges can be formatted by specifying
                               several -offset and -length pairs.
                               When only a single -offset is specified without
                               -length, clang-format will format up to the end
                               of the file.
                               Can only be used with one input file.
  --lines=<string>            - <start line>:<end line> - format a range of
                               lines (both 1-based).
                               Multiple ranges can be formatted by specifying
                               several -lines arguments.
                               Can't be used with -offset and -length.
                               Can only be used with one input file.
  -n                          - Alias for --dry-run
  --offset=<uint>             - Format a range starting at this byte offset.
                               Multiple ranges can be formatted by specifying
                               several -offset and -length pairs.
                               Can only be used with one input file.
  --output-replacements-xml  - Output replacements as XML.
  --sort-includes             - If set, overrides the include sorting behavior
                               determined by the SortIncludes style flag
  --style=<string>            - Coding style, currently supports:
                                 LLVM, Google, Chromium, Mozilla, WebKit.
                               Use -style=file to load style configuration from
                               .clang-format file located in one of the parent
                               directories of the source file (or current
                               directory for stdin).
                               Use -style="{key: value, ...}" to set specific
                               parameters, e.g.:
                                 -style="{BasedOnStyle: llvm, IndentWidth: 8}"
  --verbose                   - If set, shows the list of processed files

Generic Options:

  --help                      - Display available options (--help-hidden for more)
  --help-list                 - Display list of available options (--help-list-hidden for more)
  --version                   - Display the version of this program
```


Copyright © The Clang Team.

最后更新: 2022-07-20


Copyright © The Clang Team.

## 1.2 配置样式

`clang-format` 支持两种提供自定义样式选项的方式：直接在 `-style=` 命令行选项中指定样式配置，或者使用 `-style=file` 并将样式配置放在项目目录的 `.clang-format` 或 `_clang-format` 文件中。

当使用 `-style=file` 时，每个输入文件的 `clang-format` 将尝试在输入文件最近的父目录中找到 `.clang-format` 文件。当使用标准输入时，搜索从当前目录开始。

文件 `.clang-format` 使用 `YAML` 格式：

```
key1: value1
key2: value2
# 一条注释。
...
```

配置文件可以由几个部分组成，每个部分使用不同的语言：参数表示该配置部分针对的编程语言。有关支持的语言列表，请参阅下面语言选项的说明。第一部分可能没有设置语言，它将为所有语言设置默认样式选项。特定语言的配置节将覆盖在默认部分的设置的选项。

当 `clang-format` 格式化文件时，它使用文件名自动检测语言。当格式化标准输入或没有与其语言对应的扩展名的文件时， `-assume-filename=` 选项可以用来覆盖 `clang-format` 用于检测语言的文件名。

多语言配置文件的示例：

```
---
# 我们将使用默认的LLVM风格，但有4列缩进。
BasedOnStyle: LLVM
IndentWidth: 4
---
Language: Cpp
# 强制指向 C++ 类型的指针。
DerivePointerAlignment: false
PointerAlignment: Left
---
Language: JavaScript
# 为JS使用100列。
ColumnLimit: 100
---
Language: Proto
# 不要格式化.proto文件。
DisableFormat: true
---
Language: CSharp
# 为C#使用100列。
ColumnLimit: 100
...
```

获得包含所有预定义样式配置选项的有效 `.clang-format` 文件的一个简单方法是：

```
clang-format -style=llvm -dump-config > .clang-format
```

当在 `-style=` 选项中指定配置时，相同的配置将应用于所有输入文件。配置的格式为：

```
-style='{key1: value1, key2: value2, ...}'
```

最后更新: 2022-07-20


Copyright © The Clang Team.

## 1.3 禁用格式

Clang-format 还可以理解在带分隔符的范围内切换格式的特殊注释。注释 `// clang-format off` 或 `/* clang-format off */` 到注释 `// clang-format on` 或 `/* clang-format on */` 之间的代码不会被格式化。注释本身将被正常格式化(对齐)。

```
int formatted_code;
// clang-format off
    void    unformatted_code  ;
// clang-format on
void formatted_code_again;
```

最后更新: 2022-07-20


Copyright © The Clang Team.

# 2. 配置

## 2.1 BasedOnStyle

### 2.1.1 BasedOnStyle ( `string` )

用于配置中未特别设置的所有选项的样式。用于配置中未特别设置的所有选项的样式。

此选项仅在 **clang-format** 配置中支持 ( 在 `-style='{...}'` 和 `.clang-format` 文件 ) 。

可能的值:

- `LLVM` 一种符合 LLVM代码规范 的样式
- `Google` 一种符合 Google C++风格指南 的样式
- `Chromium` 一种符合 Chromium风格指南 的样式
- `Mozilla` 一种符合 Mozilla风格指南 的样式
- `WebKit` 一种符合 WebKit风格指南 的样式
- `Microsoft` 一种符合 Microsoft风格指南 的样式
- `GNU` 一种符合 GNU代码规范 的样式

`InheritParentConfig` 不是真正的样式，但允许使用来自父目录的 `.clang-format` 文件(如果没有父目录，则为其父目录)。如果没有找到父文件，它会退回到 `fallback` 样式，并将更改应用到该样式。使用这个选项，您可以覆盖子目录的主样式的某些部分。这也可以通过命令行实现，例如: `--style={BasedOnStyle: InheritParentConfig, ColumnLimit: 20}`

---

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.2 AccessModifierOffset

### 2.2.1 AccessModifierOffset ( `Integer` )

**Auto**

clang-format 3.3

访问修饰符的额外缩进或缩出，例如 `public:` 。

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.3 AlignAfterOpenBracket

### 2.3.1 AlignAfterOpenBracket ( `BracketAlignmentStyle` )

**Auto**

clang-format 3.8

如果为 `true`，则在左括号后水平对齐参数。

这适用于圆括号、尖括号和方括号。

可能的值：

**Align** **DontAlign** **AlwaysBreak**

`BAS_Align`：将左括号中的参数对齐，例如：

```
someLongFunction(argument1,
                 argument2);
```

`BAS_DontAlign`：不要对齐，而是使用 ContinuationIndentWidth，例如：

```
someLongFunction(argument1,
    argument2);
```

`BAS_AlwaysBreak`：如果参数不适合在一行中，则始终在左括号后中断，例如：

```
someLongFunction(
    argument1, argument2);
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.4 AlignArrayOfStructures

### 2.4.1 AlignArrayOfStructures ( `ArrayInitializerAlignmentStyle` )

**Auto**

clang-format 13

如果不是 `None`，则在对结构数组使用初始化时，会将字段对齐到列中。

可能的值：

**Left** **Right** **None**

`AIAS_Left`：对齐数组列并左对齐列，例如：

```
struct test demo[] =
{
    {56, 23,    "hello"},
    {-1, 93463, "world"},
    {7,  5,     "!!"   }
};
```

`AIAS_Right`：对齐数组列并右对齐列，例如：

```
struct test demo[] =
{
    {56,    23, "hello"},
    {-1, 93463, "world"},
    { 7,     5,    "!!"}
};
```

`AIAS_None`：不要对齐数组初始值设定项列。

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.5 AlignConsecutiveAssignments

### 2.5.1 AlignConsecutiveAssignments ( `AlignConsecutiveStyle` )

**...o**

clang-format 3.8

对齐连续赋值的样式。

`Consecutive` 会产生如下格式：

```
int a            = 1;
int somelongname = 2;
double c         = 3;
```

可能的值：

**None** **Consecutive** **AcrossEmptyLines** **AcrossComments** **AcrossEmptyLinesAndComments**

`ACS_None`：不要在连续的行上对齐赋值。

`ACS_Consecutive`：对齐连续行上的赋值。这将导致类似的格式：

```
int a            = 1;
int somelongname = 2;
double c         = 3;

int d = 3;
/* A comment. */
double e = 4;
```

`ACS_AcrossEmptyLines`：与 `ACS_Consecutive` 相同，但也跨空行，例如：

```
int a            = 1;
int somelongname = 2;
double c         = 3;

int d            = 3;
/* A comment. */
double e = 4;
```

`ACS_AcrossComments`：与 `ACS_Consecutive` 相同，但也可以跨越仅包含注释的行，例如：

```
int a            = 1;
int somelongname = 2;
double c         = 3;

int d    = 3;
/* A comment. */
double e = 4;
```

`ACS_AcrossEmptyLinesAndComments`：与 `ACS_Consecutive` 相同，但也可以跨行，只包含注释和空行，例如：

```
int a            = 1;
int somelongname = 2;
double c         = 3;

int d            = 3;
/* A comment. */
double e         = 4;
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.6 AlignConsecutiveBitFields

### 2.6.1 AlignConsecutiveBitFields ( `AlignConsecutiveStyle` )

**Auto**

clang-format 11

对齐连续位字段的样式。

将连续行的位域分隔符对齐。这将导致如下格式：

```
int aaaa : 1;
int b    : 12;
int ccc  : 8;
```

可能的值：

**None** **Consecutive** **AcrossEmptyLines** **AcrossComments** **AcrossEmptyLinesAndComments**

`ACS_None`：不要在连续的行上对齐位域。

`ACS_Consecutive`：在连续的行上对齐位域。这将导致如下格式：

```
int aaaa : 1;
int b    : 12;
int ccc  : 8;

int d : 2;
/* A comment. */
int ee : 3;
```

`ACS_AcrossEmptyLines`：与 `ACS_Consecutive` 相同，但也跨空行，例如：

```
int aaaa : 1;
int b    : 12;
int ccc  : 8;

int d    : 2;
/* A comment. */
int ee : 3;
```

`ACS_AcrossComments`：与 `ACS_Consecutive` 相同，但也可以跨越仅包含注释的行，例如：

```
int aaaa : 1;
int b    : 12;
int ccc  : 8;

int d  : 2;
/* A comment. */
int ee : 3;
```

`ACS_AcrossEmptyLinesAndComments`：与 `ACS_Consecutive` 相同，但也可以跨行，只包含注释和空行，例如：

```
int aaaa : 1;
int b    : 12;
int ccc  : 8;

int d    : 2;
/* A comment. */
int ee   : 3;
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.7 AlignConsecutiveDeclarations

## 2.7.1 AlignConsecutiveDeclarations ( `AlignConsecutiveStyle` )

**Nuo**

clang-format 3.8

对齐连续声明的样式。

将连续行的声明名称对齐。这将导致如下格式：

```
int         aaaa = 12;
float       b = 23;
std::string ccc;
```

可能的值：

**None** **Consecutive** **AcrossEmptyLines** **AcrossComments** **AcrossEmptyLinesAndComments**

`ACS_None`：不要在连续的行上对齐位声明。

`ACS_Consecutive`：在连续的行上对齐声明。这将导致如下格式：

```
int         aaaa = 12;
float       b = 23;
std::string ccc;

int a = 42;
/* A comment. */
bool c = false;
```

`ACS_AcrossEmptyLines`：与 `ACS_Consecutive` 相同，但也跨空行，例如：

```
int         aaaa = 12;
float       b = 23;
std::string ccc;

int         a = 42;
/* A comment. */
bool c = false;
```

`ACS_AcrossComments`：与 `ACS_Consecutive` 相同，但也可以跨越仅包含注释的行，例如：

```
int         aaaa = 12;
float       b = 23;
std::string ccc;

int  a = 42;
/* A comment. */
bool c = false;
```

`ACS_AcrossEmptyLinesAndComments`：与 `ACS_Consecutive` 相同，但也可以跨行，只包含注释和空行，例如：

```
int         aaaa = 12;
float       b = 23;
std::string ccc;

int         a = 42;
/* A comment. */
bool        c = false;
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.8 AlignConsecutiveMacros

### 2.8.1 AlignConsecutiveMacros ( `AlignConsecutiveStyle` )

**...o**

clang-format 9

对齐连续宏定义的样式。

`Consecutive` 会产生如下格式：

```
#define SHORT_NAME       42
#define LONGER_NAME      0x007f
#define EVEN_LONGER_NAME (2)
#define foo(x)           (x * x)
#define bar(y, z)        (y + z)
```

可能的值：

**None** **Consecutive** **AcrossEmptyLines** **AcrossComments** **AcrossEmptyLinesAndComments**

`ACS_None`：不要在连续的行上对齐宏定义。

`ACS_Consecutive`：在连续的行上对齐宏定义。这将导致如下格式：

```
#define SHORT_NAME       42
#define LONGER_NAME      0x007f
#define EVEN_LONGER_NAME (2)

#define foo(x) (x * x)
/* some comment */
#define bar(y, z) (y + z)
```

`ACS_AcrossEmptyLines`：与 `ACS_Consecutive` 相同，但也跨空行，例如：

```
#define SHORT_NAME       42
#define LONGER_NAME      0x007f
#define EVEN_LONGER_NAME (2)

#define foo(x)           (x * x)
/* some comment */
#define bar(y, z) (y + z)
```

`ACS_AcrossComments`：与 `ACS_Consecutive` 相同，但也可以跨越仅包含注释的行，例如：

```
#define SHORT_NAME       42
#define LONGER_NAME      0x007f
#define EVEN_LONGER_NAME (2)

#define foo(x)    (x * x)
/* some comment */
#define bar(y, z) (y + z)
```

`ACS_AcrossEmptyLinesAndComments`：与 `ACS_Consecutive` 相同，但也可以跨行，只包含注释和空行，例如：

```
#define SHORT_NAME       42
#define LONGER_NAME      0x007f
#define EVEN_LONGER_NAME (2)

#define foo(x)           (x * x)
/* some comment */
#define bar(y, z)        (y + z)
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.9 AlignEscapedNewlines

### 2.9.1 AlignEscapedNewlines ( `EscapedNewlineAlignmentStyle` )

**Auto**

clang-format 5

在转义的换行符中对齐反斜杠的选项。

可能的值：

**DontAlign** **Left** **Right**

`ENAS_DontAlign`：不要对齐转义的换行符。

```
#define A \
int aaaa; \
int b; \
int ddddddddd;
```

`ENAS_Left`：将转义的换行符尽可能左对齐。

```
#define A \
int aaaa; \
int b;    \
int ddddddddd;
```

`ENAS_Right`：将转义的换行符尽可能右对齐。

```
#define A                                                       \
int aaaa;                                                       \
int b;                                                          \
int ddddddddd;
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.10 AlignOperands

### 2.10.1 AlignOperands ( `OperandAlignmentStyle` )

**...o**

clang-format 12

如果为 `true`，则水平对齐二进制和三元表达式的操作数。

可能的值：

**DontAlign** **Align** **AlignAfterOperator**

`OAS_DontAlign`：不要对二进制和三元表达式的操作数进行对齐。换行后的行从行首开始缩进 ContinuationIndentWidth 空格。

`OAS_Align`：水平对齐二进制和三元表达式的操作数。

具体地说，它将一个需要在多行中分割的单个表达式的操作数对齐，例如：

```
int aaa = bbbbbbbbbbbbbb +
          ccccccccccccccc;
```

当 BreakBeforeBinaryOperators 被设置时，被包装的操作符与操作数在第一行对齐。

```
int aaa = bbbbbbbbbbbbbb
          + ccccccccccccccc;
```

`OAS_AlignAfterOperator`：水平对齐二进制和三元表达式的操作数。

这类似于 `AOS_Align` ，除了当 `BreakBeforeBinaryOperators` 被设置时，该操作符是不缩进的，以便被包装的操作数在第一行与操作数对齐。

```
int aaa = bbbbbbbbbbbbbb
        + ccccccccccccccc;
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.11 AlignTrailingComments

### 2.11.1 AlignTrailingComments ( `Boolean` )

**No**

clang-format 3.7

如果为 `true`，则对齐尾随注释。

**true** **false**

```
int a;     // My comment a
int b = 2; // comment  b

int a; // My comment a
int b = 2; // comment about b
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.12 AllowAllArgumentsOnNextLine

### 2.12.1 AllowAllArgumentsOnNextLine ( `Boolean` )

**No**

clang-format 9

如果函数调用或花括号的初始化列表不能放在一行中，则允许将所有参数放到下一行，即使 BinPackArguments 为 `false`。

**true** **false**

```
callFunction(
    a, b, c, d);

callFunction(a,
             b,
             c,
             d);
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.13 AllowAllConstructorInitializersOnNextLine

### 2.13.1 AllowAllConstructorInitializersOnNextLine (`Boolean`)

**Info**

clang-format 9

**Warning**

此选项已弃用。

请参阅 PackConstructorInitializers 的 `NextLine`。

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.14 AllowAllParametersOfDeclarationOnNextLine

### 2.14.1 AllowAllParametersOfDeclarationOnNextLine (`Boolean`)

**No**

clang-format 3.3

如果函数声明不适合一行，则允许将函数声明的所有参数放到下一行，即使 BinPackParameters 为 `false`。

**true** **false**

```
void myFunction(
    int a, int b, int c, int d, int e);

void myFunction(int a,
                int b,
                int c,
                int d,
                int e);
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.15 AllowShortBlocksOnASingleLine

### 2.15.1 AllowShortBlocksOnASingleLine ( `ShortBlockStyle` )

**Auto**

clang-format 11

依赖于值， `while (true) {continue;` 可以放在一行上。

可能的值：

**Never** **Empty** **Always**

`SBS_Never` : 永远不要将块合并为一行。

```
while (true) {
}
while (true) {
continue;
}
```

`SBS_Empty` : 只合并空块。

```
while (true) {}
while (true) {
continue;
}
```

`SBS_Always` : 总是将短块合并为一行。

```
while (true) {}
while (true) { continue; }
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.16 AllowShortCaseLabelsOnASingleLine

### 2.16.1 AllowShortCaseLabelsOnASingleLine (`Boolean`)

**Auto**

clang-format 3.6

如果为 `true`，短 `case` 标签将收缩为一行。

**true** **false**

```
switch (a) {
case 1: x = 1; break;
case 2: return;
}

switch (a) {
case 1:
  x = 1;
  break;
case 2:
  return;
}
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.17 AllowShortEnumsOnASingleLine

### 2.17.1 AllowShortEnumsOnASingleLine ( `Boolean` )

**Auto**

clang-format 12

允许在单行上使用短的枚举。

**true** **false**

```
enum { A, B } myEnum;

enum
{
    A,
    B
} myEnum;
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.18 AllowShortFunctionsOnASingleLine

## 2.18.1 AllowShortFunctionsOnASingleLine ( `ShortFunctionStyle` )

**Foo**

clang-format 3.5

根据值，`int f() { return 0; }` 可以放在一行上。

可能的值：

**None** **InlineOnly** **Empty** **Inline** **All**

`SFS_None`：永远不要将函数合并到一行中。

`SFS_InlineOnly`：只合并类中定义的函数。与 `Inline` 相同，除了它不暗含 `Empty`：即顶层空函数也不合并。

```
class Foo {
    void f() { foo(); }
};
void f() {
    foo();
}
void f() {
}
```

`SFS_Empty`：只合并空函数。

```
void f() {}
void f2() {
    bar2();
}
```

`SFS_Inline`：只合并类中定义的函数。暗含 `Empty`。

```
class Foo {
    void f() { foo(); }
};
void f() {
    foo();
}
void f() {}
```

`SFS_All`：合并所有拟合在一行上的函数。

```
class Foo {
    void f() { foo(); }
};
void f() { bar(); }
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.19 AllowShortIfStatementsOnASingleLine

### 2.19.1 AllowShortIfStatementsOnASingleLine ( `ShortIfStyle` )

**Info**

clang-format 9

如果为 `true`，则 `if (a) return;` 可以放在一条线上。

可能的值：

**Never** **WithoutElse** **OnlyFirstIf** **AllIfsAndElse**

`SIS_Never`：永远不要把简短的 `if` 放在同一行上。

```
if (a)
    return;

if (b)
    return;
else
    return;

if (c)
    return;
else {
    return;
}
```

`SIS_WithoutElse`：如果没有 `else`，只有当 `else` 不是复合语句时，才将短 `if` 放在同一行。

```
if (a) return;

if (b)
    return;
else
    return;

if (c)
    return;
else {
    return;
}
```

`SIS_OnlyFirstIf`：在同一行中放置简短的 `if`，但不要放置 `else if` 或 `else` 语句。

```
if (a) return;

if (b) return;
else if (b)
    return;
else
    return;

if (c) return;
else {
    return;
}
```

`SIS_AllIfsAndElse`：如果 `else` 不是一个复合语句，总是将简短的 `if` 放在同一行。

```
if (a) return;

if (b) return;
else return;

if (c) return;
else {
    return;
}
```


Copyright © The Clang Team.

最后更新: 2022-07-20


Copyright © The Clang Team.

# 2.20 AllowShortLambdasOnASingleLine

## 2.20.1 AllowShortLambdasOnASingleLine ( `ShortLambdaStyle` )

**Auto**

clang-format 9

依赖于值， `auto lambda []() { return 0; }` 可以放在一行上。

可能的值：

**None** **Empty** **Inline** **All**

`SLS_None`：永远不要将 `lambda` 合并到一行中。

`SLS_Empty`：只合并空的 `lambda`。

```
auto lambda = [](int a) {}
auto lambda2 = [](int a) {
    return a;
};
```

`SLS_Inline`：如果是函数的参数将 `lambda` 合并为一行。

```
auto lambda = [](int a) {
    return a;
};
sort(a.begin(), a.end(), ()[] { return x < y; })
```

`SLS_All`：将所有拟合的 `lambda` 合并到一行上。

```
auto lambda = [](int a) {}
auto lambda2 = [](int a) { return a; };
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.21 AllowShortLoopsOnASingleLine

### 2.21.1 AllowShortLoopsOnASingleLine ( `Boolean` )

**false**

clang-format 3.7

如果为 `true` ，则 `while (true) continue;` 可以放在一行上。

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.22 AlwaysBreakAfterDefinitionReturnType

### 2.22.1 AlwaysBreakAfterDefinitionReturnType ( `DefinitionReturnTypeBreakingStyle` )

**Info**

clang-format 3.7

**Warning**

该选项已弃用，为了向后兼容而保留。

函数定义返回要使用的类型中断样式。

可能的值：

**None** **All** **TopLevel**

`DRTBS_None`：自动返回类型后中断。 PenaltyReturnTypeOnItsOwnLine 被考虑在内。

`DRTBS_All`：总是在返回类型后中断。

`DRTBS_TopLevel`：总是在顶级函数的返回类型之后中断。

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.23 AlwaysBreakAfterReturnType

### 2.23.1 AlwaysBreakAfterReturnType ( `ReturnTypeBreakingStyle` )

**Auto**

clang-format 3.8

函数声明返回要使用的类型中断样式。



Copyright © The Clang Team.

可能的值：

**None** **All** **TopLevel** **AllDefinitions** **TopLevelDefinitions**

`RTBS_None`：返回类型后自动中断。PenaltyReturnTypeOnItsOwnLine 被考虑在内。

```
class A {
    int f() { return 0; };
};
int f();
int f() { return 1; }
```

`RTBS_All`：总是在返回类型之后中断。

```
class A {
int
f() {
    return 0;
};
};
int
f();
int
f() {
    return 1;
}
```

`RTBS_TopLevel`：总是在顶级函数的返回类型之后中断。

```
class A {
    int f() { return 0; };
};
int
f();
int
f() {
    return 1;
}
```

`RTBS_AllDefinitions`：总是在函数定义的返回类型之后中断。

```
class A {
int
f() {
    return 0;
};
};
int f();
int
f() {
    return 1;
}
```

`RTBS_TopLevelDefinitions`：总是在顶级定义的返回类型之后中断。

```
class A {
    int f() { return 0; };
};
int f();
int
f() {
    return 1;
}
```

---

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.24 AlwaysBreakBeforeMultilineStrings

### 2.24.1 AlwaysBreakBeforeMultilineStrings ( `Boolean` )

**false**

clang-format 3.4

如果为 `true`，总是在多行字符串字面值之前中断。

这个标志是为了使文件中有多个多行字符串的情况看起来更一致。因此，只有在此时包装字符串导致从行开始的 ContinuationIndentWidth 空格缩进时，它才会生效。

**true** **false**

```
aaaa =
    "bbbb"
    "cccc";

aaaa = "bbbb"
       "cccc";
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.25 AlwaysBreakTemplateDeclarations

### 2.25.1 AlwaysBreakTemplateDeclarations ( `BreakTemplateDeclarationsStyle` )

**...o**

clang-format 7

要使用的模板声明中断样式。

可能的值：

**No** **MultiLine** **Yes**

`BTDS_No`：在声明之前不要强制中断。PenaltyBreakTemplateDeclaration 被考虑在内。

```
template <typename T> T foo() {
}
template <typename T> T foo(int aaaaaaaaaaaaaaaaaaaaa,
                            int bbbbbbbbbbbbbbbbbbbbb) {
}
```

`BTDS_MultiLine`：只有当下列声明跨多行时，才在模板声明后强制中断。

```
template <typename T> T foo() {
}
template <typename T>
T foo(int aaaaaaaaaaaaaaaaaaaaa,
    int bbbbbbbbbbbbbbbbbbbbb) {
}
```

`BTDS_Yes`：总是在模板声明之后中断。

```
template <typename T>
T foo() {
}
template <typename T>
T foo(int aaaaaaaaaaaaaaaaaaaaa,
    int bbbbbbbbbbbbbbbbbbbbb) {
}
```

最后更新: 2022-07-20


Copyright © The Clang Team.

## 2.26 AttributeMacros

### 2.26.1 AttributeMacros ( `List of Strings` )

**Info**

clang-format 12

字符串的向量，应该被解释为属性/限定符而不是标识符。这对于语言扩展或静态分析器注释很有用。

例如：

```
x = (char *__capability)&y;
int function(void) __ununsed;
void only_writes_to_buffer(char *__output buffer);
```

在 `.clang-format` 配置文件中，可以这样配置：

```
AttributeMacros: ['__capability', '__output', '__ununsed']
```

---

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.27 BinPackArguments

## 2.27.1 BinPackArguments ( `Boolean` )

**默认**

clang-format 3.7

如果为 `false`，函数调用的参数要么都在同一行，要么各一行。

**true** **false**

```
void f() {
f(aaaaaaaaaaaaaaaaaaaa, aaaaaaaaaaaaaaaaaaaa,
    aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa);
}

void f() {
f(aaaaaaaaaaaaaaaaaaaa,
    aaaaaaaaaaaaaaaaaaaa,
    aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa);
}
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.28 BinPackParameters

### 2.28.1 BinPackParameters ( `Boolean` )

**Auto**

clang-format 3.7

如果为 `false`，函数声明或函数定义的形参要么都在同一行，要么各有一行。

**true** **false**

```
void f(int aaaaaaaaaaaaaaaaaaaa, int aaaaaaaaaaaaaaaaaaaa,
    int aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa) {}

void f(int aaaaaaaaaaaaaaaaaaaa,
    int aaaaaaaaaaaaaaaaaaaa,
    int aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa) {}
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.29 BitFieldColonSpacing

## 2.29.1 BitFieldColonSpacing ( `BitFieldColonSpacingStyle` )

**Auto**

clang-format 12

用于位字段的 `BitFieldColonSpacingStyle`。

可能的值：

**Both** **None** **Before** **After**

`BFCS_Both`：每边加一个空格。

```
unsigned bf : 2;
```

`BFCS_None`：在 `:` 周围不添加空格(除非需要 AlignConsecutiveBitFields)。

```
unsigned bf:2;
```

`BFCS_Before`：只在 `:` 前面添加空格。

```
unsigned bf :2;
```

`BFCS_After`：只在 `:` 后面加空格(如果 AlignConsecutiveBitFields 需要的话，可以在前面加空格)。

```
unsigned bf: 2;
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.30 BraceWrapping

## 2.30.1 BraceWrapping ( `BraceWrappingFlags` )

**Nno**

clang-format 3.8

控制大括号换行情况。

如果 BreakBeforeBraces 被设置为 `BS_Custom` ，使用这个来指定应该如何处理每个单独的大括号。否则，这将被忽略。

```
# Example of usage:
BreakBeforeBraces: Custom
BraceWrapping:
  AfterEnum: true
  AfterStruct: false
  SplitEmptyFunction: false
```

嵌套的配置标记：

- `bool AfterCaseLabel` : `case` 标签的括号

**true** **false**

```
switch (foo) {
    case 1:
    {
        bar();
        break;
    }
    default:
    {
        plop();
    }
}

switch (foo) {
    case 1: {
        bar();
        break;
    }
    default: {
        plop();
    }
}
```

- `bool AfterClass` : `class` 定义的括号

**true** **false**

```
class foo {};

class foo
{};
```

- `BraceWrappingAfterControlStatementStyle AfterControlStatement` ：包含控制语句 ( `if` / `for` / `while` / `switch` /...)



Copyright © The Clang Team.

可能的值：

Never　MultiLine　Always

`BWACS_Never`：不要在控制语句后面用大括号括起来。

```
if (foo()) {
} else {
}
for (int i = 0; i < 10; ++i) {
}
```

`BWACS_MultiLine`：在多行控制语句后只使用大括号。

```
if (foo && bar &&
    baz)
{
    quux();
}
while (foo || bar) {
}
```

`BWACS_Always`：在控制语句后面总是用大括号括起来。

```
if (foo())
{
} else
{}
for (int i = 0; i < 10; ++i)
{}
```

- `bool AfterEnum` : `enum` 定义的括号

true　false

```
enum X : int
{
    B
};

enum X : int { B };
```

- `bool AfterFunction`：函数定义的括号

true　false

```
void foo()
{
    bar();
    bar2();
}

void foo() {
    bar();
    bar2();
}
```


Copyright © The Clang Team.

`bool AfterNamespace` : `namespace` 定义的括号

**true** **false**

```
namespace
{
    int foo();
    int bar();
}

namespace {
    int foo();
    int bar();
}
```

- `bool AfterObjCDeclaration` : `ObjC` 定义的括号 (interfaces, implementations...). `@autoreleasepool` 和 `@synchronized` 块根据 AfterControlStatement 标志进行包装
- `bool AfterStruct` : `struct` 定义的括号

**true** **false**

```
struct foo
{
    int x;
};

struct foo {
    int x;
};
```

- `bool AfterUnion` : `union` 定义的括号

**true** **false**

```
union foo
{
    int x;
}

union foo {
    int x;
}
```

- `bool AfterExternBlock` : `extern` 定义的括号

**true** **false**

```
extern "C"
{
int foo();
}

extern "C" {
int foo();
}
```

- `bool BeforeCatch` : `catch` 之前的括号

**true** **false**

```
try {
    foo();
}
catch () {
}

try {
    foo();
} catch () {
}
```

- `bool BeforeElse` : `else` 之前的括号


Copyright © The Clang Team.

**true** **false**

```
if (foo()) {
}
else {
}

if (foo()) {
} else {
}
```

- `bool BeforeLambdaBody` : `lambda` 块的括号

**true** **false**

```
connect(
    []()
    {
        foo();
        bar();
    });

connect([]() {
    foo();
    bar();
});
```

- `bool BeforeWhile` : `while` 前的括号

**true** **false**

```
do {
    foo();
}
while (1);

do {
    foo();
} while (1);
```

- `bool IndentBraces` ：缩进包装的括号本身
- `bool SplitEmptyFunction` ：如果为 `false` ，则空函数体可以放在一行上。该选项仅在函数的左括号已经被包装的情况下使用，即设置了 AfterFunction 括号封装模式，并且该函数不可以/不应该放在单行上(如 AllowShortFunctionsOnASingleLine 和构造函数格式化选项)。

**true** **false**

```
int f()
{
}

int f()
{}
```

- `bool SplitEmptyRecord` ：如果为 `false` ，空记录(例如类，结构体或联合)主体可以放在一行上。此选项仅在记录的开括号已经被包装时使用，即设置了 AfterClass (对于类)括号封装模式。

**true** **flase**

```
class Foo
{
}

class Foo
{}
```

- `bool SplitEmptyNamespace` ：如果为 `false` ，则空的命名空间主体可以放在一行上。该选项仅在名称空间的左大括号已经被包装时使用，即设置了 `AfterNamespace` 大括号包装模式。


Copyright © The Clang Team.

**true** **false**

```
namespace Foo
{
}

namespace Foo
{}
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.31 BreakAfterJavaFieldAnnotations

### 2.31.1 BreakAfterJavaFieldAnnotations ( `Boolean` )

**Auto**

clang-format 3.8

在 `Java` 文件中字段的每个注释之后中断。

**true** **false**

```
@Partial
@Mock
DataLoad loader;

@Partial @Mock DataLoad loader;
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.32 BreakBeforeBinaryOperators

## 2.32.1 BreakBeforeBinaryOperators ( `BinaryOperatorStyle` )

**Auto**

clang-format 3.6

二元运算符换行的方式。

可能的值：

**None** **NonAssignment** **All**

`BOS_None`：运算符之后中断。

```
LoooooooooooongType looooooooooooooooooooooongVariable =
    someLoooooooooooooooooongFunction();

bool value = aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa +
                     aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa ==
                 aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa &&
             aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa >
                 ccccccccccccccccccccccccccccccccccccccccccc;
```

`BOS_NonAssignment`：在非赋值操作符之前中断。

```
LoooooooooooongType looooooooooooooooooooooongVariable =
    someLoooooooooooooooooongFunction();

bool value = aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
                     + aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
                 == aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
             && aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
                     > ccccccccccccccccccccccccccccccccccccccccccc;
```

`BOS_All`：运算符之前中断。

```
LoooooooooooongType looooooooooooooooooooooongVariable
    = someLoooooooooooooooooongFunction();

bool value = aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
                     + aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
                 == aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
             && aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
                     > ccccccccccccccccccccccccccccccccccccccccccc;
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.33 BreakBeforeBraces

### 2.33.1 BreakBeforeBraces ( `BraceBreakingStyle` )

**Auto**

clang-format 3.7

要使用的大括号中断样式。



Copyright © The Clang Team.

可能的值:


Copyright © The Clang Team.

**Attach** **Linux** **Mozilla** **Stroustrup** **Allman** **Whitesmiths** **GNU** **WebKit** **Custom**

`BS_Attach`：始终将大括号附加到周围的上下文。

```
namespace N {
enum E {
  E1,
  E2,
};

class C {
public:
  C();
};

bool baz(int i) {
  try {
    do {
      switch (i) {
      case 1: {
        foobar();
        break;
      }
      default: {
        break;
      }
      }
    } while (--i);
    return true;
  } catch (...) {
    handleError();
    return false;
  }
}

void foo(bool b) {
  if (b) {
    baz(2);
  } else {
    baz(5);
  }
}

void bar() { foo(true); }
} // namespace N
```

`BS_Linux`：与 `Attach` 类似，但在函数、名称空间和类定义的大括号前中断。

```
namespace N
{
enum E {
  E1,
  E2,
};

class C
{
public:
  C();
};

bool baz(int i)
{
  try {
    do {
      switch (i) {
      case 1: {
        foobar();
        break;
      }
      default: {
        break;
      }
      }
    } while (--i);
    return true;
  } catch (...) {
    handleError();
    return false;
  }
}

void foo(bool b)
{
  if (b) {
    baz(2);
  } else {
    baz(5);
  }
}

void bar() { foo(true); }
} // namespace N
```

`BS_Mozilla`：与 `Attach` 类似，但在枚举、函数和记录定义的大括号前中断。


Copyright © The Clang Team.

```
namespace N {
enum E
{
```

最后更新: 2022-07-20


Copyright © The Clang Team.

## 2.34 BreakBeforeConceptDeclarations

### 2.34.1 BreakBeforeConceptDeclarations ( `Boolean` )

**false**

clang-format 13

如果为 `true`， `concept` 将被放置到新行。

**true** **false**

```
template<typename T>
concept ...

template<typename T> concept ...
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.35 BreakBeforeTernaryOperators

## 2.35.1 BreakBeforeTernaryOperators ( `Boolean` )

**false**

clang-format 3.7

如果为 `true`，将在换行符之后放置三元操作符。

**true** **false**

```
veryVeryVeryVeryVeryVeryVeryVeryVeryVeryVeryLongDescription
    ? firstValue
    : SecondValueVeryVeryVeryVeryLong;

veryVeryVeryVeryVeryVeryVeryVeryVeryVeryVeryLongDescription ?
    firstValue :
    SecondValueVeryVeryVeryVeryLong;
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.36 BreakConstructorInitializers

## 2.36.1 BreakConstructorInitializers ( `BreakConstructorInitializersStyle` )

**Auto**

clang-format 5

要使用的构造函数初始化式样式。

可能的值：

**BeforeColon** **BeforeComma** **AfterColon**

`BCIS_BeforeColon`：在冒号之前和逗号之后中断构造函数初始化式。

```
Constructor()
    : initializer1(),
    initializer2()
```

`BCIS_BeforeComma`：在冒号和逗号之前中断构造函数初始化式，并将逗号与冒号对齐。

```
Constructor()
    : initializer1()
    , initializer2()
```

`BCIS_AfterColon`：在冒号和逗号后中断构造函数初始化式。

```
Constructor() :
    initializer1(),
    initializer2()
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.37 BreakInheritanceList

## 2.37.1 BreakInheritanceList ( `BreakInheritanceListStyle` )

**Auto**

clang-format 7

要使用的继承列表样式。

可能的值：

**BeforeColon** **BeforeComma** **AfterColon**

`BILS_BeforeColon`：在冒号之前和逗号之后中断继承列表。

```
class Foo
    : Base1,
    Base2
{};
```

`BILS_BeforeComma`：在冒号和逗号之前中断继承列表，并将逗号与冒号对齐。

```
class Foo
    : Base1
    , Base2
{};
```

`BILS_AfterColon`：在冒号和逗号后断开继承列表。

```
class Foo :
    Base1,
    Base2
{};
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.38 BreakStringLiterals

### 2.38.1 BreakStringLiterals ( `Boolean` )

**Auto**

clang-format 3.9

允许在格式化时中断字符串文字。

**true** **false**

```
const char* x = "veryVeryVeryVeryVeryVe"
                "ryVeryVeryVeryVeryVery"
                "VeryLongString";

const char* x =
    "veryVeryVeryVeryVeryVeryVeryVeryVeryVeryVeryVeryLongString";
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.39 ColumnLimit

### 2.39.1 ColumnLimit ( `Unsigned` )

**Info**

clang-format 3.7

列的限制。

列限制为 `0` 意味着没有列限制。在这种情况下， `clang-format` 将在语句中尊重输入的断行决定，除非它们与其他规则相抵触。

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.40 CommentPragmas

### 2.40.1 CommentPragmas ( `String` )

**Info**

clang-format 3.7

一个正则表达式，它描述具有特殊含义的注释，不应该将注释分成行或以其他方式更改注释。

```
// CommentPragmas: '^ FOOBAR pragma:'
// Will leave the following line unaffected
#include <vector> // FOOBAR pragma: keep
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.41 CompactNamespaces

## 2.41.1 CompactNamespaces ( `Boolean` )

**Auto**

clang-format 5

如果为 `true`，连续的名称空间声明将在同一行上。如果为 `false`，则每个名称空间都声明在一个新的行中。

**true** **false**

```
namespace Foo { namespace Bar {
}}

namespace Foo {
namespace Bar {
}
}
```

如果它不能放在一行中，溢出的名称空间将被包装：

```
namespace Foo { namespace Bar {
namespace Extra {
}}}
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.42 ConstructorInitializerAllOnOneLineOrOnePerLine

### 2.42.1 ConstructorInitializerAllOnOneLineOrOnePerLine ( `Boolean` )

**Info**

clang-format 3.7

**Warning**

此选项已弃用。

请参阅 PackConstructorInitializers 的 `CurrentLine`。

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.43 ConstructorInitializerIndentWidth

### 2.43.1 ConstructorInitializerIndentWidth ( `Unsigned` )

**Auto**

clang-format 3.7

用于缩进构造函数初始化列表和继承列表的字符数。

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.44 ContinuationIndentWidth

### 2.44.1 ContinuationIndentWidth ( `Unsigned` )

**Info**

clang-format 3.7

**连续行缩进宽度。**

```
ContinuationIndentWidth: 2
```

```
int i =         //  VeryVeryVeryVeryVeryLongComment
  longFunction( // Again a long comment
    arg);
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.45 Cpp11BracedListStyle

## 2.45.1 Cpp11BracedListStyle ( `Boolean` )

**Auto**

clang-format 3.4

如果为 `true`，将带大括号的列表格式化为最适合 `C++11` 带大括号的列表。

重要区别：

- 括号内没有空格。
- 在右括号之前不能换行。
- 使用延续缩进，而不是使用块缩进。

基本上，带大括号的 `C++11` 列表的格式与函数调用在其位置上的格式完全相同。如果带括号的列表跟在一个名称后面(例如，类型名或变量名)，`clang-format` 的格式就好像 `{}` 是带有该名称的函数调用的圆括号一样。如果没有名称，则假定名称长度为零。

**true** **false**

```
vector<int> x{1, 2, 3, 4};
vector<T> x{{}, {}, {}, {}};
f(MyMap[{composite, key}]);
new int[3]{1, 2, 3};

vector<int> x{ 1, 2, 3, 4 };
vector<T> x{ {}, {}, {}, {} };
f(MyMap[{ composite, key }]);
new int[3]{ 1, 2, 3 };
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.46 DeriveLineEnding

### 2.46.1 DeriveLineEnding ( `Boolean` )

**Auto**

clang-format 11

分析格式化文件中最常用的行结束符( `\r\n` 或 `\n` )。只有当没有派生项时，UseCRLF 才用作后备。

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.47 DerivePointerAlignment

### 2.47.1 DerivePointerAlignment ( `Boolean` )

**Auto**

clang-format 3.7

如果为 `true`，分析格式化文件，以确定最常见的 `&` 和 `*` 对齐方式。指针和引用对齐样式将根据文件中找到的首选项进行更新。然后，PointerAlignment 仅用作后备。

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.48 DisableFormat

### 2.48.1 DisableFormat ( `Boolean` )

**No**

clang-format 3.7

完全禁用格式。

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.49 EmptyLineAfterAccessModifier

## 2.49.1 EmptyLineAfterAccessModifier ( `EmptyLineAfterAccessModifierStyle` )

**Info**

clang-format 14

定义何时在访问修饰符后放置空行。 `EmptyLineBeforeAccessModifier` 配置处理两个访问修饰符之间的空行数。

可能的值：

**Never** **Leave** **Always**

`ELAAMS_Never`：删除访问修饰符后的所有空行。

```
struct foo {
private:
  int i;
protected:
  int j;
  /* comment */
public:
  foo() {}
private:
protected:
};
```

`ELAAMS_Leave`：在访问修饰符之后保留现有的空行。应用 MaxEmptyLinesToKeep。

`ELAAMS_Always`：如果没有访问修饰符，请始终在访问修饰符后添加空行。 应用 MaxEmptyLinesToKeep。

```
struct foo {
private:

  int i;
protected:

  int j;
  /* comment */
public:

  foo() {}
private:

protected:

};
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.50 EmptyLineBeforeAccessModifier

## 2.50.1 EmptyLineBeforeAccessModifier (`EmptyLineBeforeAccessModifierStyle`)

**Info**

clang-format 13

定义在何种情况下在访问修饰符之前放置空行。

可能的值：

**Never** **Leave** **LogicalBlock** **Always**

`ELBAMS_Never`：删除访问修饰符之前的所有空行。

```
struct foo {
private:
  int i;
protected:
  int j;
  /* comment */
public:
  foo() {}
private:
protected:
};
```

`ELBAMS_Leave`：在访问修饰符之前保持现有的空行。

`ELBAMS_LogicalBlock`：只有当访问修饰符开始一个新的逻辑块时才添加空行。逻辑块是由一个或多个成员字段或函数组成的一组。

```
struct foo {
private:
  int i;

protected:
  int j;
  /* comment */
public:
  foo() {}

private:
protected:
};
```

`ELBAMS_Always`：总是在访问修饰符之前添加空行，除非访问修饰符位于结构或类定义的开头。

```
struct foo {
private:
  int i;

protected:
  int j;
  /* comment */

public:
  foo() {}

private:

protected:
};
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.51 ExperimentalAutoDetectBinPacking

### 2.51.1 ExperimentalAutoDetectBinPacking ( `Boolean` )

**Info**

clang-format 3.7

如果为 `true`，`clang-format` 将检测函数调用和定义是否每行使用一个参数进行格式化。

每个调用都可以被包含，每行或不确定。如果它是不确定的，例如完全在一行上，但需要做出决定，`clang-format` 分析输入文件中是否有其他包含情况，并相应地采取行动。

**Note**

这是一个实验性的标志，它可能会消失或被重命名。不要在配置文件中使用它，等等。使用时自负风险。

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.52 FixNamespaceComments

### 2.52.1 FixNamespaceComments ( `Boolean` )

**No**

clang-format 5

如果为 `true`，`clang-format` 将为短名称空间添加缺失的名称空间结束注释，并修复无效的现有名称空间。短的由 ShortNamespaceLines 控制。

**true** **false**

```
namespace a {
    foo();
    bar();
} // namespace a

namespace a {
    foo();
    bar();
}
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.53 ForEachMacros

### 2.53.1 ForEachMacros ( `List of Strings` )

**Info**

clang-format 3.7

宏的矢量，应该解释为 `foreach` 循环，而不是函数调用。

这些应该是以下形式的宏：

```
FOREACH(<variable-declaration>, ...)
  <loop-body>
```

在 `.clang-format` 配置文件中，可以这样配置：

```
ForEachMacros: ['RANGES_FOR', 'FOREACH']
```

例如：`BOOST_FOREACH`

---

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.54 IfMacros

### 2.54.1 IfMacros ( `List of Strings` )

**Info**

clang-format 14

应解释为条件而不是函数调用的宏向量。

这些期望是以下形式的宏：

```
IF(...)
  <conditional-body>
else IF(...)
  <conditional-body>
```

在 `.clang-format` 配置文件中，可以这样配置：

```
IfMacros: ['IF']
```

例如： `KJ_IF_MAYBE`

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.55 IncludeBlocks

## 2.55.1 IncludeBlocks ( `IncludeBlocksStyle` )

**Info**

clang-format 7

根据这个值，可以将多个 `#include` 块按类别排序。

可能的值：

**Preserve** **Merge** **Regroup**

`IBS_Preserve`：分别对每个 `#include` 块进行排序。

```
#include "b.h"              into      #include "b.h"

#include <lib/main.h>                 #include "a.h"
#include "a.h"                        #include <lib/main.h>
```

`IBS_Merge`：将多个 `#include` 块合并在一起，并按一个排序。

```
#include "b.h"              into      #include "a.h"
                                      #include "b.h"
#include <lib/main.h>                 #include <lib/main.h>
#include "a.h"
```

`IBS_Regroup`：将多个 `#include` 块合并在一起，并按一个排序。然后根据类别优先级分组。参考 IncludeCategories。

```
#include "b.h"              into      #include "a.h"
                                      #include "b.h"
#include <lib/main.h>
#include "a.h"                        #include <lib/main.h>
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.56 IncludeCategories

### 2.56.1 IncludeCategories ( `List of IncludeCategories` )

**Info**

clang-format 7

正则表达式表示用于排序 `#include` 的不同 `#include` 类别。

支持 POSIX 扩展正则表达式。

这些正则表达式按顺序匹配包含的文件名(包括 `<>` 或 `""`)。分配属于第一个匹配正则表达式的值，`#include` 首先根据类别编号递增排序，然后在每个类别内按字母顺序排序。

如果所有正则表达式都不匹配，则 `INT_MAX` 被分配为类别。源文件的主标题自动获得类别 `0`。因此它通常保存在 `#include` (https://llvm.org/docs/CodingStandards.html#include-style) 的开头。然而，如果你有某些总是需要优先的头，你也可以分配负优先级。

当 `IncludeBlocks = IBS_Regroup` 时，可以使用第三个可选字段 `SortPriority` 来定义 `#include` 应该被排序的优先级。`Priority` 的值定义了 `#include` 块的顺序，也允许对不同优先级的 `#include` 块进行分组。如果没有分配 `SortPriority`，则将其设置为 `Priority` 的默认值。

每个正则表达式都可以用大小写敏感字段标记为区分大小写，但默认情况下它不是。

要在 `.clang-format` 文件中配置它，请使用：

```
IncludeCategories:
  - Regex:           '^"(llvm|llvm-c|clang|clang-c)/'
    Priority:        2
    SortPriority:    2
    CaseSensitive:   true
  - Regex:           '^(<|"(gtest|gmock|isl|json)/)'
    Priority:        3
  - Regex:           '<[[:alnum:].]+>'
    Priority:        4
  - Regex:           '.*'
    Priority:        1
    SortPriority:    0
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.57 IncludeIsMainRegex

### 2.57.1 IncludeIsMainRegex ( `String` )

**Info**

clang-format 7

指定在 `file-to-main-include` 映射中允许的后缀的正则表达式。

当猜测 `#include` 是否是 `main` `include` 时(为类别 `0` 赋值，见上面)，使用这个允许后缀的正则表达式到标题干。完成部分匹配，因此:

- "" 表示 “任意后缀”
- "$" 表示 “没有后缀”

例如，如果配置为 `(_test)?$`，则头文件 `a.h` 将被视为 `a.cc` 和 `a_test.cc` 中的 `main`。

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.58 IncludeIsMainSourceRegex

### 2.58.1 IncludeIsMainSourceRegex ( `String` )

**Info**

clang-format 7

为在 `file-to-main-include` 映射中允许被视为 `main` 的格式化文件指定一个正则表达式。

默认情况下，只有当文件以: `.c` ， `.cc` ， `.cpp` ， `.c++` ， `.cxx` ， `.m` 或 `.mm` 扩展名结束时， `clang-format` 才会将文件视为 `main` 。对于这些文件，会猜测 `main` include(要分配类别 `0` ，参见上面)。这个配置选项允许附加后缀和扩展名的文件被视为 `main` 。

例如，如果这个选项被配置为 `(Impl\.hpp)$` ，那么一个文件 `ClassImpl.hpp` 被认为是 `main` (除了 `Class.c` 、 `Class.cc` 、 `Class.cpp` 等)和 `main include file` 逻辑将被执行(稍后阶段还将使用 IncludeIsMainRegex 设置)。如果不设置此选项， `ClassImpl.hpp` 不会把主要的包含文件放在任何其他包含文件之前。

---

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.59 IndentAccessModifiers

### 2.59.1 IndentAccessModifiers ( `Boolean` )

**Auto**

clang-format 13

指定访问修饰符是否应该有自己的缩进级别。

当为 `false` 时，访问修饰符相对于记录成员缩进(或退缩)，表示 AccessModifierOffset。记录成员缩进比记录低一级。当为 `true` 时，访问修饰符将获得自己的缩进级别。因此，记录成员总是比记录缩进 `2` 级，而不管是否存在访问修饰符。忽略 AccessModifierOffset 的值。

**true** **fasle**

```
class C {
    class D {
        void bar();
        protected:
        D();
    };
    public:
        C();
};
void foo() {
    return 1;
}

class C {
    class D {
        void bar();
    protected:
        D();
    };
public:
    C();
};
void foo() {
    return 1;
}
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.60 IndentCaseBlocks

## 2.60.1 IndentCaseBlocks ( `Boolean` )

**Auto**

clang-format 11

缩进 `case` 标签阻塞 `case` 标签的一级。

当为 `false` 时， `case` 标签后面的块使用与 `case` 标签相同的缩进级别，将 `case` 标签视为if语句。当为 `true` 时，该块作为范围块缩进。

**true** **false**

```
switch (fool) {
case 1:
  {
    bar();
  }
  break;
default:
  {
    plop();
  }
}

switch (fool) {
case 1: {
  bar();
} break;
default: {
  plop();
}
}
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.61 IndentCaseLabels

### 2.61.1 IndentCaseLabels ( `Boolean` )

**Info**

clang-format 3.3

缩进 `case` 标记 `switch` 语句的一个级别。

当为 `false` 时，使用与 `switch` 语句相同的缩进级别。 `switch` 语句体总是比 `case` 标签缩进一级( `case` 标签后面的第一个块除外，它本身会缩进代码——除非启用了 IndentCaseBlocks)。

**true** **false**

```
switch (fool) {
  case 1:
    bar();
    break;
  default:
    plop();
}

switch (fool) {
case 1:
    bar();
    break;
default:
    plop();
}
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.62 IndentExternBlock

## 2.62.1 IndentExternBlock ( `IndentExternBlockStyle` )

**Info**

clang-format 12

`IndentExternBlockStyle` 是 `extern` 块的缩进类型。

可能的值：

**AfterExternBlock** **NoIndent** **Indent**

`IEBS_AfterExternBlock`：向后兼容 `AfterExternBlock` 的缩进。

```
IndentExternBlock: AfterExternBlock
BraceWrapping.AfterExternBlock: true
```

```
extern "C"
{
    void foo();
}
```

```
IndentExternBlock: AfterExternBlock
BraceWrapping.AfterExternBlock: false
```

```
extern "C" {
void foo();
}
```

`IEBS_NoIndent`：不缩进外部块。

```
extern "C" {
void foo();
}
```

`IEBS_Indent`：缩进外部块。

```
extern "C" {
    void foo();
}
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.63 IndentGotoLabels

## 2.63.1 IndentGotoLabels ( `Boolean` )

**Auto**

clang-format 10

缩进 `goto` 标签。

当为 `false` 时， `goto` 标签被推到最左侧。

**true** **false**

```
int f() {
    if (foo()) {
    label1:
        bar();
}
label2:
    return 1;
}

int f() {
    if (foo()) {
label1:
        bar();
}
label2:
    return 1;
}
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.64 IndentPPDirectives

### 2.64.1 IndentPPDirectives ( `PPDirectiveIndentStyle` )

**Info**

clang-format 6

要使用的预处理器指令缩进样式。

可能的值：

**None** **AfterHash** **BeforeHash**

`PPDIS_None`：不缩进任何指令。

```
#if FOO
#if BAR
#include <foo>
#endif
#endif
```

`PPDIS_AfterHash`：缩进 `#` 后的指令。

```
#if FOO
#  if BAR
#    include <foo>
#  endif
#endif
```

`PPDIS_BeforeHash`：在 `#` 前缩进指令。

```
#if FOO
  #if BAR
    #include <foo>
  #endif
#endif
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.65 IndentRequires

## 2.65.1 IndentRequires ( `Boolean` )

**Auto**

clang-format 13

在模板中缩进 `requires` 子句

**true** **false**

```
template <typename It>
    requires Iterator<It>
void sort(It begin, It end) {
//....
}

template <typename It>
requires Iterator<It>
void sort(It begin, It end) {
//....
}
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.66 IndentWidth

IndentWidth ( `Unsigned` )

**...o**

clang-format 3.7

用于缩进的列数。

```
IndentWidth: 3
```

```
void f() {
   someFunction();
   if (true, false) {
      f();
   }
}
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.67 IndentWrappedFunctionNames

## 2.67.1 IndentWrappedFunctionNames ( `Boolean` )

**Auto**

clang-format 3.7

如果函数定义或声明包装在类型之后，则缩进。

**true** **false**

```
LoooooooooooooooooooooooooooooooooooooooongReturnType
    LoooooooooooooooooooooooooooooongFunctionDeclaration();

LoooooooooooooooooooooooooooooooooooooooongReturnType
LoooooooooooooooooooooooooooooongFunctionDeclaration();
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.68 InsertTrailingCommas

## 2.68.1 InsertTrailingCommas ( `TrailingCommaStyle` )

**...o**

clang-format 12

如果设置为 `TCS_Wrapped`，则会在跨多行换行的容器文本(数组和对象)中插入末尾的逗号。它目前仅对 `JavaScript` 可用，并在默认情况下禁用 `TCS_None`。 `InsertTrailingCommas` 不能与 BinPackArguments 一起使用，因为插入逗号会禁用包装。

```
const someArray = [
aaaaaaaaaaaaaaaaaaaaaaaaaa,
aaaaaaaaaaaaaaaaaaaaaaaaaa,
aaaaaaaaaaaaaaaaaaaaaaaaaa,
//                        ^ inserted
]
```

可能的值：

**None** **Wrapped**

`TCS_None`：不要在后面插入逗号。

`TCS_Wrapped`：在包装在多行中的括号字面值中插入尾随逗号。请注意，这在概念上与装箱不兼容，因为末尾的逗号被用作指示容器应该每行格式化(即不包含)。因此，插入末尾逗号会抵消括号的作用。

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.69 JavaImportGroups

### 2.69.1 JavaImportGroups ( `List of Strings` )

**Info**

clang-format 8

一个由 `Java` 导入所需的组排序的前缀向量。

一个组的前缀可以是另一个组的子集——总是匹配最长的前缀。在组中，导入按字典顺序排序。静态导入被单独分组，并遵循相同的分组规则。默认情况下，静态导入被放置在非静态导入之前，但是这个行为会被另一个选项 SortJavaStaticImport 更改。

在 `.clang-format` 配置文件中，可以像下面的 `yaml` 示例中那样配置。这将导致导入被格式化，如下面的 `Java` 示例所示。

```yaml
JavaImportGroups: ['com.example', 'com', 'org']
```

```java
import static com.example.function1;

import static com.test.function2;

import static org.example.function3;

import com.example.ClassA;
import com.example.Test;
import com.example.a.ClassB;

import com.test.ClassC;

import org.example.ClassD;
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.70 JavaScriptQuotes

## 2.70.1 JavaScriptQuotes (JavaScriptQuoteStyle)

**Info**

clang-format 3.9

`JavaScript` 字符串使用的 `JavaScriptQuoteStyle`。

可能的值:

**Leave** **Single** **Double**

`JSQS_Leave`: 字符串引号保持原样。

```
string1 = "foo";
string2 = 'bar';
```

`JSQS_Single`: 总是使用单引号。

```
string1 = 'foo';
string2 = 'bar';
```

`JSQS_Double`: 总是使用双引号。

```
string1 = "foo";
string2 = "bar";
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.71 JavaScriptWrapImports

### 2.71.1 JavaScriptWrapImports ( `boolean` )

**false**

clang-format 3.9

是否包装 `JavaScript` 的导入/导出语句。

**true** **false**

```
import {
    VeryLongImportsAreAnnoying,
    VeryLongImportsAreAnnoying,
    VeryLongImportsAreAnnoying,
} from 'some/module.js'

import {VeryLongImportsAreAnnoying, VeryLongImportsAreAnnoying, VeryLongImportsAreAnnoying,} from "some/module.js"
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.72 KeepEmptyLinesAtTheStartOfBlocks

### 2.72.1 KeepEmptyLinesAtTheStartOfBlocks ( `Boolean` )

**false**

clang-format 3.7

如果为 `true`，则保留块开头的空行。

**true** **false**

```
if (foo) {

    bar();
}

if (foo) {
    bar();
}
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.73 LambdaBodyIndentation

### 2.73.1 LambdaBodyIndentation ( `LambdaBodyIndentationKind` )

**Auto**

clang-format 13

`lambda` 主体的缩进样式。（默认）导致 `lambda` 主体相对于签名的缩进级别缩进一个额外的级别。 `OuterScope` 强制 `lambda` 主体相对于包含 `lambda` 签名的父作用域缩进一个额外的级别。 对于回调繁重的代码，将签名缩进两级并使用 `OuterScope` 可能会提高可读性。KJ 风格指南需要 `OuterScope` 。KJ风格指南

**Signature** **OuterScope**

`LBI_Signature`：相对于 `lambda` 签名对齐 `lambda` 主体。这是默认设置。

```
someMethod(
    [](SomeReallyLongLambdaSignatureArgument foo) {
      return;
    });
```

`LBI_OuterScope`：相对于 `lambda` 签名所在的外部范围的缩进级别对齐 `lambda` 主体。

```
someMethod(
    [](SomeReallyLongLambdaSignatureArgument foo) {
  return;
});
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.74 Language

### 2.74.1 Language ( `LanguageKind` )

**Info**

clang-format 3.5

语言，这种格式风格是针对的。

可能的值：

**None** **Cpp** **CSharp** **Java** **JavaScript** **ObjC** **Proto** **TableGen** **TextProto**

`LK_None`：不使用。

`LK_Cpp`：应用于 `C`，`C++`。

`LK_CSharp`：应该用于 `C#`。

`LK_Java`：应该用于 `Java`。

`LK_JavaScript`：应该用于 `JavaScript`。

`LK_ObjC`：应该用于 `Objective-C`， `Objective-C++`。

`LK_Proto`：应该用于协议缓冲区 (https://developers.google.com/protocol-buffers)。

`LK_TableGen`：应该用于 `TableGen` 代码。

`LK_TextProto`：应该用于文本格式的协议缓冲区消息 (https://developers.google.com/protocol-buffers)。

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.75 MacroBlockBegin

## 2.75.1 MacroBlockBegin ( `String` )

**Info**

clang-format 3.7

匹配开始块的宏的正则表达式。

```
MacroBlockBegin: "^NS_MAP_BEGIN|\
NS_TABLE_HEAD$"
MacroBlockEnd: "^\
NS_MAP_END|\
NS_TABLE_.*_END$"
```

**With** **Without**

```
NS_MAP_BEGIN
  foo();
NS_MAP_END

NS_TABLE_HEAD
  bar();
NS_TABLE_FOO_END

NS_MAP_BEGIN
foo();
NS_MAP_END

NS_TABLE_HEAD
bar();
NS_TABLE_FOO_END
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.76 MacroBlockEnd

### 2.76.1 MacroBlockEnd ( `String` )

**Info**

clang-format 3.7

匹配以块结束的宏的正则表达式。

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.77 MaxEmptyLinesToKeep

### 2.77.1 MaxEmptyLinesToKeep (`Unsigned`)

**Auto**

clang-format 3.7

要保留的最大连续空行数。

```
MaxEmptyLinesToKeep: 1         vs.     MaxEmptyLinesToKeep: 0
```

```
int f() {
  int = 1;

  i = foo();

  return i;
}
```

```
int f() {
    int i = 1;
    i = foo();
    return i;
}
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.78 NamespaceIndentation

### 2.78.1 NamespaceIndentation ( `NamespaceIndentationKind` )

**Info**

clang-format 3.7

名称空间使用的缩进。

可能的值：

**None** **Inner** **All**

`NI_None`：不要在名称空间中缩进。

```
namespace out {
int i;
namespace in {
int i;
}
}
```

`NI_Inner`：只在内部名称空间中缩进(嵌套在其他名称空间中)。

```
namespace out {
int i;
namespace in {
    int i;
}
}
```

`NI_All`：缩进所有名称空间。

```
namespace out {
    int i;
    namespace in {
        int i;
    }
}
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.79 NamespaceMacros

### 2.79.1 NamespaceMacros ( `List of Strings` )

**Info**

clang-format 9

用于打开命名空间块的宏的向量。

这些应该是以下形式的宏：

```
NAMESPACE(<namespace-name>, ...) {
  <namespace-content>
}
```

例如：`TESTSUITE`

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.80 ObjCBinPackProtocolList

### 2.80.1 ObjCBinPackProtocolList ( `BinPackStyle` )

**Auto**

clang-format 7

控制包装 `Objective-C` 协议一致性列表项到尽可能少的行，当他们超过 ColumnLimit。

如果是 `Auto` (默认值)，则委托 `BinPackParameters` 中的值。如果是这样的话，把符合 `Objective-C` 协议的列表项放入尽可能少的行中，只要它们超过 ColumnLimit。

如果是 `Always` ，那么当 `Objective-C` 协议一致性列表项超过 ColumnLimit 时，总是把它们打包成尽可能少的行。

如果是 `Never` ，当超过 ColumnLimit 时，将符合 `Objective-C` 协议的列表项放在单独的行上。

- Always (或 `Auto` , `BinPackParameters=true` ):

```
@interface ccccccccccccc () <
    ccccccccccccc, ccccccccccccc,
    ccccccccccccc, ccccccccccccc> {
}
```

- Never (或 `Auto` , `BinPackParameters=false` ):

```
@interface ddddddddddddd () <
    ddddddddddddd,
    ddddddddddddd,
    ddddddddddddd,
    ddddddddddddd> {
}
```

可能的值:

- `BPS_Auto` : 自动确定参数装箱行为。
- `BPS_Always` : 总是包装参数。
- `BPS_Never` : 从不包装参数

最后更新: 2022-07-20


Copyright © The Clang Team.

## 2.81 ObjCBlockIndentWidth

### 2.81.1 ObjCBlockIndentWidth ( `Unsigned` )

**Info**

clang-format 3.7

用于 `ObjC` 块缩进的字符数。

```
ObjCBlockIndentWidth: 4
```

```
[operation setCompletionBlock:^{
    [self onOperationDone];
}];
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.82 ObjCBreakBeforeNestedBlockParam

## 2.82.1 ObjCBreakBeforeNestedBlockParam ( `Boolean` )

**Yes**

clang-format 12

当函数调用中有嵌套的块形参时，将形参列表分成几行。

**true** **false**

```
(void)_aMethod
{
    [self.test1 t:self
                w:self
        callback:^(typeof(self) self, NSNumber *u, NSNumber *v) {
            u = c;
        }]
}

(void)_aMethod
{
    [self.test1 t:self w:self callback:^(typeof(self) self, NSNumber
    *u, NSNumber *v) {
        u = c;
    }]
}
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.83 ObjCSpaceAfterProperty

### 2.83.1 ObjCSpaceAfterProperty ( `Boolean` )

**No**

clang-format 3.7

在 `Objective-C` 的 `@property` 后面加一个空格，即使用 `@property (readonly)` 而不是 `@property(readonly)`。

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.84 ObjCSpaceBeforeProtocolList

### 2.84.1 ObjCSpaceBeforeProtocolList ( `Boolean` )

**Yes**

clang-format 3.7

在 `Objective-C` 协议列表前面添加一个空格，例如使用 `Foo <protocol>` 代替 `Foo<protocol>`。

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.85 PPIndentWidth

## 2.85.1 PPIndentWidth ( `Integer` )

**Info**

clang-format 14

用于缩进预处理器语句的列数。 当设置为 -1（默认）时，IndentWidth 也用于预处理器语句。

```
PPIndentWidth: 1
```

```
#ifdef __linux__
# define FOO
#else
# define BAR
#endif
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.86 PackConstructorInitializers

## 2.86.1 PackConstructorInitializers ( `PackConstructorInitializersStyle` )

**Info**

clang-format 14

要使用的包构造函数初始值设定项样式。

可能的值：

**Never** **BinPack** **CurrentLine** **NextLine**

`PCIS_Never`：始终将每个构造函数初始化器放在自己的行上。

```
Constructor()
    : a(),
      b()
```

`PCIS_BinPack`：Bin-pack 构造函数初始值设定项。

```
Constructor()
    : aaaaaaaaaaaaaaaaaaaa(), bbbbbbbbbbbbbbbbbbbb(),
      cccccccccccccccccccc()
```

`PCIS_CurrentLine`：如果合适，将所有构造函数初始值设定项放在当前行。 否则，将每个都放在自己的行上。

```
Constructor() : a(), b()

Constructor()
    : aaaaaaaaaaaaaaaaaaaa(),
      bbbbbbbbbbbbbbbbbbbb(),
      ddddddddddddd()
```

`PCIS_NextLine`：与 `PCIS_CurrentLine` 相同，但如果所有构造函数初始值设定项都不适合当前行，则尝试将它们放在下一行。

```
Constructor() : a(), b()

Constructor()
    : aaaaaaaaaaaaaaaaaaaa(), bbbbbbbbbbbbbbbbbbbb(), ddddddddddddd()

Constructor()
    : aaaaaaaaaaaaaaaaaaaa(),
      bbbbbbbbbbbbbbbbbbbb(),
      cccccccccccccccccccc()
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.87 PenaltyBreakAssignment

### 2.87.1 PenaltyBreakAssignment ( `Unsigned` )

**No**

clang-format 5

突破赋值运算符的补偿。

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.88 PenaltyBreakBeforeFirstCallParameter

### 2.88.1 PenaltyBreakBeforeFirstCallParameter (`Unsigned`)

**Auto**

clang-format 3.7

`call(` 后中断函数调用的补偿。

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.89 PenaltyBreakComment

### 2.89.1 PenaltyBreakComment ( `Unsigned` )

**Auto**

clang-format 3.7

注释中引入的每个换行符的补偿。

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.90 PenaltyBreakFirstLessLess

### 2.90.1 PenaltyBreakFirstLessLess ( `Unsigned` )

**Auto**

clang-format 3.7

在第一个 `<<` 之前中断的补偿。

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.91 PenaltyBreakString

### 2.91.1 PenaltyBreakString ( `Unsigned` )

**Auto**

clang-format 3.7

在字符串文字中引入的每个换行符的补偿。

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.92 PenaltyBreakTemplateDeclaration

### 2.92.1 PenaltyBreakTemplateDeclaration (`Unsigned`)

**Auto**

clang-format 7

模板声明后中断的补偿。

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.93 PenaltyExcessCharacter

### 2.93.1 PenaltyExcessCharacter (`Unsigned`)

**Auto**

clang-format 3.7

列限制之外的每个字符的补偿。

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.94 PenaltyIndentedWhitespace

### 2.94.1 PenaltyIndentedWhitespace ( `Unsigned` )

**Auto**

clang-format 12

空格缩进的每个字符的补偿（相对于前导非空格列计算）。

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.95 PenaltyReturnTypeOnItsOwnLine

### 2.95.1 PenaltyReturnTypeOnItsOwnLine (Unsigned)

**No**

clang-format 3.7

将函数的返回类型放在它自己的行上的补偿。

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.96 PointerAlignment

### 2.96.1 PointerAlignment ( `PointerAlignmentStyle` )

**Auto**

clang-format 3.7

指针和引用对齐样式。

可能的值：

**Left** **Right** **Middle**

`PAS_Left`：将指针向左对齐。

```
int* a;
```

`PAS_Right`：将指针向右对齐。

```
int *a;
```

`PAS_Middle`：在中间对齐指针。

```
int * a;
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.97 QualifierAlignment

## 2.97.1 QualifierAlignment ( `QualifierAlignmentStyle` )

**Info**

clang-format 14

排列说明符和限定符的不同方式（例如 `const` / `volatile` ）。

**Waring**

将 `QualifierAlignment` 设置为 `Leave` 以外的其他内容，可能会导致错误的代码格式，这是由于 `clang-format` 缺乏完整的语义信息而做出的错误决定。因此，应格外小心地审查使用此选项所做的代码更改。

可能的值：

**Leave** **Left** **Right** **Custom**

`QAS_Leave`：不要将说明符/限定符更改为左对齐或右对齐（默认）。

```
int const a;
const int *a;
```

`QAS_Left`：将说明符/限定符更改为左对齐。

```
const int a;
const int *a;
```

`QAS_Right`：将说明符/限定符更改为右对齐。

```
int const a;
int const *a;
```

`QAS_Custom`：更改说明符/限定符以根据 QualifierOrder 对齐。

```
QualifierOrder: ['inline', 'static' , 'type', 'const']
```

```
int const a;
int const *a;
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.98 QualifierOrder

### 2.98.1 QualifierOrder ( `List of Strings` )

**Info**

clang-format 14

限定符出现的顺序。顺序是一个数组，可以包含以下任何一项：

- const
- inline
- static
- constexpr
- volatile
- restrict
- type

**Attention**

它必须包含 `type`。 `type` 左侧的项目将放置在类型的左侧并按提供的顺序对齐。 `type` 右侧的项目将放置在类型的右侧并按照提供的顺序对齐。

```
QualifierOrder: ['inline', 'static', 'type', 'const', 'volatile' ]
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.99 RawStringFormats

## 2.99.1 RawStringFormats ( `List of RawStringFormats` )

**Nio**

clang-format 6

定义在原始字符串中检测支持的语言代码块的提示。

带有匹配的分隔符或包含匹配的函数名的原始字符串将根据 `.clang-format` 文件中定义的语言的样式进行重新格式化，假设该语言是指定的语言。如果在 `.clang-format` 文件中没有为特定语言定义样式，则使用 BasedOnStyle 给出的预定义样式。如果没有找到 BasedOnStyle，则格式化基于 `llvm` 风格。匹配分隔符优先于匹配的封闭函数名，以确定原始字符串内容的语言。

如果指定了规范分隔符，同一语言中出现的其他分隔符将尽可能更新为规范。

每种语言最多应该有一个规范，每个分隔符和包围函数不应该出现在多个规范中。

要在 `.clang-format` 文件中配置它，请使用：

```
RawStringFormats:
  - Language: TextProto
      Delimiters:
        - 'pb'
        - 'proto'
      EnclosingFunctions:
        - 'PARSE_TEXT_PROTO'
      BasedOnStyle: google
  - Language: Cpp
      Delimiters:
        - 'cc'
        - 'cpp'
      BasedOnStyle: llvm
      CanonicalDelimiter: 'cc'
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.100 ReferenceAlignment

## 2.100.1 ReferenceAlignment ( `ReferenceAlignmentStyle` )

**Auto**

clang-format 14

引用对齐样式（覆盖引用的 PointerAlignment）。

可能的值：

**Pointer** **Left** **Right** **Middle**

`RAS_Pointer`：像 PointerAlignment 一样对齐引用。

`RAS_Left`：将引用对齐到左侧。

```
int& a;
```

`RAS_Right`：将引用对齐到右侧。

```
int &a;
```

`RAS_Middle`：在中间对齐引用。

```
int & a;
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.101 ReflowComments

## 2.101.1 ReflowComments ( `Boolean` )

**Auto**

clang-format 4

如果为 `true`，`clang-format` 将尝试重新输入注释。

**false** **true**

```
// veryVeryVeryVeryVeryVeryVeryVeryVeryVeryVeryLongComment with plenty of information
/* second veryVeryVeryVeryVeryVeryVeryVeryVeryVeryVeryLongComment with plenty of information */

// veryVeryVeryVeryVeryVeryVeryVeryVeryVeryVeryLongComment with plenty of
// information
/* second veryVeryVeryVeryVeryVeryVeryVeryVeryVeryVeryLongComment with plenty of
* information */
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.102 ShortNamespaceLines

## 2.102.1 ShortNamespaceLines ( `Unsigned` )

**Auto**

clang-format 14

短名称空间所跨的未换行的最大行数。默认为 `1`。

这通过计算未换行的行(即不包含打开或关闭名称空间大括号)来确定短名称空间的最大长度，并使 FixNamespaceComments 省略为这些行添加的结束注释。

```
ShortNamespaceLines: 1     vs.     ShortNamespaceLines: 0
```

```
namespace a {                      namespace a {
  int foo;                           int foo;
}                                  } // namespace a
```

```
ShortNamespaceLines: 1     vs.     ShortNamespaceLines: 0
```

```
namespace b {                      namespace b {
  int foo;                           int foo;
  int bar;                           int bar;
} // namespace b                   } // namespace b
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.103 SortIncludes

## 2.103.1 SortIncludes (SortIncludesOptions)

**Info**

clang-format 4

控制 `clang-format` 是否以及如何排序 `#include`。如果没有，则包含永远不会排序。如果不区分大小写，则包含以 `ascii` 或不区分大小写的方式排序。如果区分大小写，则包含按字母或区分大小写的方式排序。

可能的值：

**Never** **CaseSensitive** **CaseInsensitive**

`SI_Never`：包含永远不会排序。

```
#include "B/A.h"
#include "A/B.h"
#include "a/b.h"
#include "A/b.h"
#include "B/a.h"
```

`SI_CaseSensitive`：`include` 以大小写敏感的方式排序。

```
#include "A/B.h"
#include "A/b.h"
#include "B/A.h"
#include "B/a.h"
#include "a/b.h"
```

`SI_CaseInsensitive`：`include` 以不区分字母或大小写的方式排序。

```
#include "A/B.h"
#include "A/b.h"
#include "a/b.h"
#include "B/A.h"
#include "B/a.h"
```

最后更新: 2022-07-20


Copyright © The Clang Team.

# 2.104 SortJavaStaticImport

## 2.104.1 SortJavaStaticImport ( `SortJavaStaticImportOptions` )

**Info**

clang-format 12

在对 `Java` 导入排序时，默认情况下，静态导入放在非静态导入之前。如果 JavaStaticImportAfterImport 在后面，则静态导入放在非静态导入之后。

可能的值：

**Before** **After**

`SJSIO_Before`：静态导入放在非静态导入之前。

```
import static org.example.function1;

import org.example.ClassA;
```

`SJSIO_After`：静态导入放在非静态导入之后。

```
import org.example.ClassA;

import static org.example.function1;
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.105 SortUsingDeclarations

## 2.105.1 SortUsingDeclarations ( `Boolean` )

**Info**

clang-format 5

如果为 `true`， `clang-format` 将使用声明进行排序。

使用声明的顺序定义如下:使用 `::` 分隔字符串并丢弃任何初始化的空字符串。每个列表的最后一个元素是一个非命名空间名称;其他的都是命名空间名称。按字典顺序对名称列表进行排序，其中个人名称的排序顺序是所有非名称空间名称出现在所有名称空间名称之前，并且在这些组中，名称不区分大小写。

**true** **false**

```
using std::cin;
using std::cout;

using std::cout;
using std::cin;
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.106 SpaceAfterCStyleCast

### 2.106.1 SpaceAfterCStyleCast ( `Boolean` )

**Yes**

clang-format 3.5

如果为 `true`，则在 `C` 风格强制转换后插入一个空格。

**true** **false**

```
(int) i;

(int)i;
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.107 SpaceAfterLogicalNot

## 2.107.1 SpaceAfterLogicalNot ( `Boolean` )

**Yes**

clang-format 9

如果为 `true`，则在逻辑否操作符(`!`)后面插入一个空格。

**true** **false**

```
! someExpression();
```

```
!someExpression();
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.108 SpaceAfterTemplateKeyword

### 2.108.1 SpaceAfterTemplateKeyword ( `Boolean` )

**No**

clang-format 4

如果为 `true`，则在 `template` 关键字后面插入一个空格。

**true** **false**

```
template <int> void foo();

template<int> void foo();
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.109 SpaceAroundPointerQualifiers

## 2.109.1 SpaceAroundPointerQualifiers ( `SpaceAroundPointerQualifiersStyle` )

**Info**

clang-format 12

定义在何种情况下在指针限定符之前或之后放置空格

可能的值：

**Default** **Before** **After** **Both**

`SAPQ_Default`：不要确保指针限定符周围有空格，而是使用 PointerAlignment。

```
PointerAlignment: Left                PointerAlignment: Right

void* const* x = NULL;         vs.    void *const *x = NULL;
```

`SAPQ_Before`：确保在指针限定符之前有空格。

```
PointerAlignment: Left                PointerAlignment: Right

void* const* x = NULL;         vs.    void * const *x = NULL;
```

`SAPQ_After`：确保在指针限定符后有空格。

```
PointerAlignment: Left                PointerAlignment: Right

void* const * x = NULL;         vs.    void *const *x = NULL;
```

`SAPQ_Both`：确保在指针限定符的前后都有空格。

```
PointerAlignment: Left                PointerAlignment: Right

void* const * x = NULL;         vs.    void * const *x = NULL;
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.110 SpaceBeforeAssignmentOperators

## 2.110.1 SpaceBeforeAssignmentOperators ( `Boolean` )

**Auto**

clang-format 3.7

如果为 `false`，则在赋值操作符之前删除空格。

**true** **false**

```
int a = 5;
a += 42;

int a= 5;
a+= 42;
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.111 SpaceBeforeCaseColon

## 2.111.1 SpaceBeforeCaseColon ( `Boolean` )

**Auto**

clang-format 12

如果为 `false`，则在 `case` 冒号之前删除空格。

**true** **false**

```
switch (x) {
case 1 : break;
}

switch (x) {
case 1: break;
}
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.112 SpaceBeforeCpp11BracedList

## 2.112.1 SpaceBeforeCpp11BracedList ( `Boolean` )

**Auto**

clang-format 7

如果为 `true`，则在用于初始化对象的 `C++11` 大括号列表之前插入一个空格(在前面的标识符或类型之后)。

**true** **false**

```
Foo foo { bar };
Foo {};
vector<int> { 1, 2, 3 };
new int[3] { 1, 2, 3 };

Foo foo{ bar };
Foo{};
vector<int>{ 1, 2, 3 };
new int[3]{ 1, 2, 3 };
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.113 SpaceBeforeCtorInitializerColon

### 2.113.1 SpaceBeforeCtorInitializerColon ( `Boolean` )

**Yes**

clang-format 7

如果为 `false`，在构造函数初始化器冒号之前的空格将被删除。

**true** **false**

```
Foo::Foo() : a(a) {}

Foo::Foo(): a(a) {}
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.114 SpaceBeforeInheritanceColon

## 2.114.1 SpaceBeforeInheritanceColon ( `Boolean` )

**Yes**

clang-format 7

如果为 `false`，在继承冒号之前的空格将被删除。

**true** **false**

```
class Foo : Bar {}
```

```
class Foo: Bar {}
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.115 SpaceBeforeParens

## 2.115.1 SpaceBeforeParens ( `SpaceBeforeParensOptions` )

**Auto**

clang-format 3.5

定义在何种情况下在开括号前放空格。

可能的值：

**Never** **ControlStatements** **ControlStatementsExceptForEachMacros** **NonEmptyParentheses** **Always**

`SBPO_Never`：不要在括号前放空格。

```
void f() {
    if(true) {
        f();
    }
}
```

`SBPO_ControlStatements`：只在控制语句关键字( `for` / `if` / `while` ...)之后的开括号前放一个空格。

```
void f() {
    if (true) {
        f();
    }
}
```

`SBPO_ControlStatementsExceptForEachMacros`：与 `SBPO_ControlStatements` 相同，除了这个选项不适用于 ForEach 宏。这在 ForEach 宏被视为函数调用而不是控制语句的项目中非常有用。

```
void f() {
    Q_FOREACH(...) {
        f();
    }
}
```

`SBPO_NonEmptyParentheses`：只有括号不是空的时候才在括号前放一个空格，即 `()`

```
void() {
    if (true) {
        f();
        g (x, y, z);
    }
}
```

`SBPO_Always`：总是在开括号前放一个空格，除非语法规则禁止(在类似函数的宏定义中)或由其他风格规则决定(在一元操作符、开括号后，等等)。

```
void f () {
    if (true) {
        f ();
    }
}
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.116 SpaceBeforeParensOptions

## 2.116.1 SpaceBeforeParensOptions ( `SpaceBeforeParensCustom` )

**Nno**

clang-format 14

控制括号前的单个空格。

如果 SpaceBeforeParens 设置为 `Custom` ，则使用它来指定应如何处理括号大小写前的每个单独的空格。否则，这将被忽略。

用法示例：

```
SpaceBeforeParens: Custom
SpaceBeforeParensOptions:
  AfterControlStatements: true
  AfterFunctionDefinitionName: true
Nested configuration flags:
```

- `bool AfterControlStatements` : 如果为 `true` ，则在控制语句关键字 ( `for` / `if` / `while` ... ) 和左括号之间放置空格。

```
true:                                   false:
if (...) {}                     vs.     if(...) {}
```

- `bool AfterForeachMacros` : 如果为 `true` ，则在 `foreach` 宏和左括号之间放置空格。

```
true:                                   false:
FOREACH (...)                   vs.     FOREACH(...)
  <loop-body>                             <loop-body>
```

- `bool AfterFunctionDeclarationName` : 如果为 `true` ，则在函数声明名称和左括号之间放置一个空格。

```
true:                                   false:
void f ();                      vs.     void f();
```

- `bool AfterFunctionDefinitionName` : 如果为 `true` ，则在函数定义名称和左括号之间放置一个空格。

```
true:                                   false:
void f () {}                    vs.     void f() {}
```

- `bool AfterIfMacros` : 如果为 `true` ，则在 `if` 宏和左括号之间放置空格。

```
true:                                   false:
IF (...)                        vs.     IF(...)
  <conditional-body>                      <conditional-body>
```

- `bool BeforeNonEmptyParentheses` : 如果为 `true` ，则仅当括号不为空时才在括号前放置一个空格。

```
true:                                   false:
void f (int a);                 vs.     void f();
f (a);                                  f();
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.117 SpaceBeforeRangeBasedForLoopColon

## 2.117.1 SpaceBeforeRangeBasedForLoopColon ( `Boolean` )

**Auto**

clang-format 7

如果为 `false`，在基于范围的 `for loop` 冒号之前将删除空格。

**true** **false**

```
for (auto v : values) {}
```

```
for(auto v: values) {}
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.118 SpaceBeforeSquareBrackets

## 2.118.1 SpaceBeforeSquareBrackets ( `Boolean` )

**Auto**

clang-format 11

如果为 `true`，空格将在 `[` 之前。不受影响。只有第一个 `[` 会被加一个空格。

**true** **false**

```
int a [5];
int a [5][5];

int a[5];
int a[5][5];
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.119 SpaceInEmptyBlock

## 2.119.1 SpaceInEmptyBlock ( `Boolean` )

**Info**

clang-format 11

如果为 `true`，将在 `{}` 中插入空格。

**true** **false**

```
void f() { }
while (true) { }

void f() {}
while (true) {}
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.120 SpaceInEmptyParentheses

### 2.120.1 SpaceInEmptyParentheses (`Boolean`)

**Auto**

clang-format 3.7

如果为 `true`，可以在 `()` 中插入空格。

**true** **false**

```
void f( ) {
    int x[] = {foo( ), bar( )};
    if (true) {
        f( );
    }
}

void f() {
    int x[] = {foo(), bar()};
    if (true) {
        f();
    }
}
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.121 SpacesBeforeTrailingComments

## 2.121.1 SpacesBeforeTrailingComments (`Unsigned`)

**Info**

clang-format 3.7

尾行注释(`//` -注释)之前的空格数。

这不会影响末尾的块注释(`/*` -注释)，因为这些注释通常有不同的使用模式和一些特殊情况。

```
SpacesBeforeTrailingComments: 3
```

```
void f() {
    if (true) {   // foo1
        f();      // bar
    }             // foo
}
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.122 SpacesInAngles

## 2.122.1 SpacesInAngles ( `SpacesInAnglesStyle` )

**Auto**

clang-format 14

用于模板参数列表的 `SpacesInAnglesStyle`。

可能的值：

**Never** **Always** **Leave**

`SIAS_Never`：删除 `<` 之后和 `>` 之前的空格。

```
static_cast<int>(arg);
std::function<void(int)> fct;
```

`SIAS_Always`：在 `<` 之后和 `>` 之前添加空格。

```
static_cast< int >(arg);
std::function< void(int) > fct;
```

`SIAS_Leave`：如果存在空格，则在 `<` 之后和之前保留一个空格。 选项标准： `Cpp03` 优先。

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.123 SpacesInCStyleCastParentheses

### 2.123.1 SpacesInCStyleCastParentheses ( `Boolean` )

**Auto**

clang-format 3.7

如果为 `true`，则可以在 `C` 风格的强制转换中插入空格。

**true** **false**

```
x = ( int32 )y
```

```
x = (int32)y
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.124 SpacesInConditionalStatement

## 2.124.1 SpacesInConditionalStatement ( `Boolean` )

**Auto**

clang-format 11

如果为 `true`，则在 `if` / `for` / `switch` / `while` 条件周围插入空格。

**true** **false**

```
if ( a )  { ... }
while ( i < 5 )  { ... }

if (a) { ... }
while (i < 5) { ... }
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.125 SpacesInContainerLiterals

## 2.125.1 SpacesInContainerLiterals ( `Boolean` )

**Auto**

clang-format 3.7

如果为 `true`，则在容器字面量(例如 `ObjC` 和 `Javascript` 数组和 `dict` 字面量)中插入空格。

**true** **false**

```
var arr = [ 1, 2, 3 ];
f({a : 1, b : 2, c : 3});

var arr = [1, 2, 3];
f({a: 1, b: 2, c: 3});
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.126 SpacesInLineCommentPrefix

## 2.126.1 SpacesInLineCommentPrefix ( `SpacesInLineComment` )

**Info**

clang-format 14

一行注释的开头允许有多少个空格。要禁用最大值，请将其设置为 `-1`，但最大值优先于最小值。

```
Minimum = 1 Maximum = -1 // 强制输入一个空格

// 但是更多的空间是可能的

Minimum = 0 Maximum = 0 //强制每个注释直接在斜杠后面开始
```

请注意，在行注释部分，后续行的相对缩进被保留，这意味着如下：

```
before:                                 after:
Minimum: 1
//if (b) {                              // if (b) {
//  return true;                        //   return true;
//}                                     // }

Maximum: 0
/// List:                              ///List:
///  - Foo                             /// - Foo
///    - Bar                           ///   - Bar
```

嵌套的配置标记：

- `unsigned Minimum` 注释开始处的最小空格数。
- `unsigned Maximum` 注释开始处的最大空格数。

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.127 SpacesInParentheses

## 2.127.1 SpacesInParentheses ( `Boolean` )

**Auto**

clang-format 3.7

如果为 `true`，则在前后插入空格。

**true** **false**

```
t f( Deleted & ) & = delete;

t f(Deleted &) & = delete;
```

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.128 SpacesInSquareBrackets

### 2.128.1 SpacesInSquareBrackets ( `Boolean` )

**false**

clang-format 3.7

如果为 `true`，将在 `[` 和之前 `]` 后面插入空格。没有参数或未指定大小的数组声明的匿名函数将不受影响。

**true** **false**

```
int a[ 5 ];
std::unique_ptr<int[]> foo() {} // Won't be affected

int a[5];
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.129 Standard

## 2.129.1 Standard ( `LanguageStandard` )

**Auto**

clang-format 3.7

解析和格式化与这个标准兼容的 `C++` 结构。

**c++03** **latest**

```
vector<set<int> > x;

vector<set<int>> x;
```

可能的值：

**Cpp03** **Cpp11** **Cpp14** **Cpp17** **Cpp20** **Latest** **Auto**

`LS_Cpp03`：解析和格式化为 `C++03`。 `Cpp03` 是 `c++03` 的一个已弃用别名

`LS_Cpp11`：解析和格式化为 `C++11`。

`LS_Cpp14`：解析和格式化为 `C++14`。

`LS_Cpp17`：解析和格式化为 `C++17`。

`LS_Cpp20`：解析和格式化为 `C++20`。

`LS_Latest`：使用最新支持的语言版本进行解析和格式化。 `Cpp11` 是最新的一个已弃用别名。

`LS_Auto`：基于输入自动检测。

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.130 StatementAttributeLikeMacros

## 2.130.1 StatementAttributeLikeMacros ( `List of Strings` )

**Info**

clang-format 12

在语句前面被忽略的宏，就像它们是一个属性一样。这样它们就不会被解析为标识符，例如 `Qts emit` 。

```
AlignConsecutiveDeclarations: true
StatementAttributeLikeMacros: []
```

```
unsigned char data = 'x';
emit          signal(data); //被解析为变量声明。
```

```
AlignConsecutiveDeclarations: true
StatementAttributeLikeMacros: [emit]
```

```
unsigned char data = 'x';
emit signal(data); //现在又好了。
```

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.131 StatementMacros

## 2.131.1 StatementMacros ( `List of Strings` )

**Info**

clang-format 8

宏的一个向量，应该被解释为完整的语句。

典型的宏是表达式，需要添加分号;有时情况并非如此，这允许 `clang-format` 能够识别这种情况。

例如: `Q_UNUSED`

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.132 TabWidth

## 2.132.1 TabWidth ( `Unsigned` )

**Info**

clang-format 3.7

用于制表位的列数。

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.133 TypenameMacros

### 2.133.1 TypenameMacros ( `List of Strings` )

**Also**

clang-format 3.7

宏的一个向量，它应该被解释为类型声明而不是函数调用。

这些应该是以下形式的宏：

```
STACK_OF(...)
```

在 `.clang-format` 配置文件中，可以这样配置：

```
TypenameMacros: ['STACK_OF', 'LIST']
```

例如：`OpenSSL STACK_OF`，`BSD LIST_ENTRY`

---

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.134 UseCRLF

## 2.134.1 UseCRLF ( `Boolean` )

**Auto**

clang-format 11

用 `\r\n` 代替 `\n` 换行。如果 (DeriveLineEnding)(../DeriveLineEnding) 为 `true`，也用作回退。

最后更新: 2022-07-20



Copyright © The Clang Team.

# 2.135 UseTab

## 2.135.1 UseTab ( `UseTabStyle` )

**Info**

clang-format 3.7

在结果文件中使用制表符的方法。

可能的值：

**Never** **ForIndentation** **ForContinuationAndIndentation** **AlignWithSpaces** **Always**

`UT_Never`：从不使用制表符。

`UT_ForIndentation`：仅在缩进时使用制表符。

`UT_ForContinuationAndIndentation`：用制表符填充所有的前导空格，并在一行中使用空格进行对齐(例如连续的赋值和声明)。

`UT_AlignWithSpaces`：使用制表符进行行延续和缩进，使用空格进行对齐。

`UT_Always`：当我们需要填补至少从一个制表位到下一个制表位的空白时，使用制表位。

最后更新: 2022-07-20



Copyright © The Clang Team.

## 2.136 WhitespaceSensitiveMacros

### 2.136.1 WhitespaceSensitiveMacros ( `List of Strings` )

**Info**

clang-format 12

一个宏向量，它是空格敏感的，不应该被触及。

这些应该是以下形式的宏：

```
STRINGIZE(...)
```

在 `.clang-format` 配置文件中，可以这样配置：

```
WhitespaceSensitiveMacros: ['STRINGIZE', 'PP_STRINGIZE']
```

例如：`BOOST_PP_STRINGIZE`。

---

最后更新: 2022-07-20



Copyright © The Clang Team.